Range, Exceptions, Testing and File-Handling

COMP3522 OBJECT ORIENTED PROGRAMMING 2

WEEK 4

RANGE

What’s a range?

It represents an immutable sequence of numbers

It is commonly used for looping in a *for loop*

Stores a few values:

1. Start value
2. Stop value
3. Step value.

That’s it!

So no matter how big the range is, the range object that stores it is *just three values*.

This is very memory efficient

It’s easy to create lists of numbers

Python’s range( ) function can help us create lists composed of sequences of numbers:

```
list(range(10)) #[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
list(range(1, 11)) #[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
list(range(0, 30, 5)) #[0, 5, 10, 15, 20, 25]
```

It’s easy to create lists of numbers

Note the relationship between the arguments passed to the function and the list that is actually created:

```
list(range(0, -10, -1)) #[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
list(range(0)) #[]
list(range(5,0)) #[]
list(range(5,0,-1)) #[5, 4, 3, 2, 1]
```

More about the range data type

```
r = range(0, 20, 2)
print(r) #range(0, 20, 2)
print(type(r)) #<class 'range’>

#r would generate: 0, 2, 4, 6, 8, 10, 12, 14, 16, 18
```

Range contains useful functions that provide:

1. Containment tests (in) — `print(10 in r)`
2. Element index lookup — `print(r.index(10))`
3. Slicing — `print(list(r[2:5]))`
4. Support for negative indices. — `print(r[-1])`

Range example code

```
r = range(0, 20, 2)
print(r)
range(0, 20, 2)
print(11 in r)
False
print(10 in r)
True
print(r.index(10)) #which index is 10 at?
5
```

```
print(r[5])
10
print(r[:5]) #get first 5 index values
[0, 2, 4, 6, 8]
print(r[-1]) #get last value
18
```

```
#r would generate: 0, 2, 4, 6, 8, 10, 12, 14, 16, 18
```

We can find the min and the max easily

```
digits = range(1, 11)
print(digits)
range(1,11)
print(max(digits))
10
print(min(digits))
1
print(sum(digits))
55
letters = list('Python > Java')
sum(letters) # <-- <-- Will this work?
```

Errors and Exceptions

TRY RAISE CATCH ELSE FINALLY

Errors and Exceptions

In all our attempts to write encapsulated and polymorphic OOP code, we tend to forget about all the edge cases and ways our program can fail.

Like many other languages, Python raises Errors and exceptions that notify the developer of what went wrong, and where.

Errors usually provide **useful messages** and a **stacktrace** showing all the line numbers and function calls involved.

```
Traceback (most recent call last):
  File "D:/Google Drive/BCIT Work Synced Folder/My Courses/COMP3522 OOP 2/Fall 2019/Week 5/Quiz Code/question one.py", line 19, in <module>
    main()
  File "D:/Google Drive/BCIT Work Synced Folder/My Courses/COMP3522 OOP 2/Fall 2019/Week 5/Quiz Code/question one.py", line 16, in main
    print(f'Salary: {employee_account_data["invalidkey"]}')
  File "D:/Google Drive/BCIT Work Synced Folder/My Courses/COMP3522 OOP 2/Fall 2019/Week 5/Quiz Code/question one.py", line 11, in __getitem__
    return self.data[item]
KeyError: 'invalidkey'
```

Exceptions

Open up the python command shell and try some of this code:

```
>>> user_choice = int(input(“Enter Menu Option”))
>>> Hello
```

```
ValueError: invalid literal for int() with base 10: 'hello'
```

```
>>> my_list = [1,2,3]
>>> my_list + ‘4’
```

```
TypeError: can only concatenate list (not "str") to list
```

```
>>> my_list.add(4)
```

```
AttributeError: 'list' object has no attribute 'add‘
```

Exceptions

```
>>> 9/0
ZeroDivisionError: division by zero
```

```
>>> print “99”  # This used to work in Python 2
SyntaxError: Missing parentheses in call to 'print'. Did you mean print("99")?
```

```
>>> print(my_variable)
NameError: name 'my_variable' is not defined
```

Common Exceptions and Errors

- AttributeError
- FloatingPointError
- IndexError
- KeyError
- SystemExit
- NotImplementedError
- NameError
- ZeroDivisionError
- SyntaxError
- TypeError
- StopIteration
- ValueError
- AssertionError
- MemoryError

All of these inherit from the BaseException class.

Throwing an exception - **raise**

In languages like Java or C++, we're probably used to **throwing exceptions** with the keyword **throw**

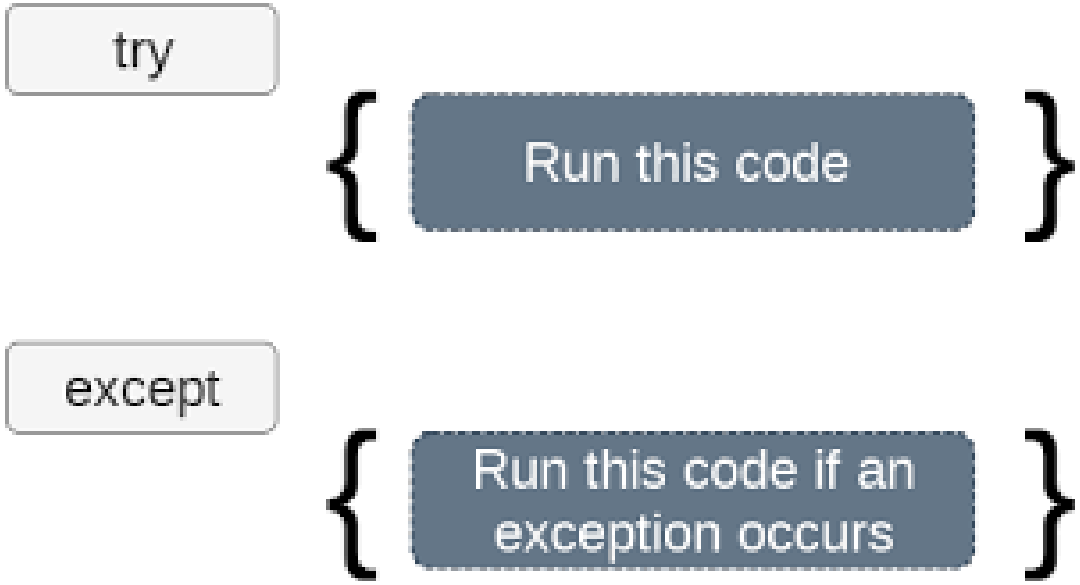
Python uses a different keyword, **raise**

Functionally does the same thing

```
def divide(num1, num2):
    if num2 == 0:
        raise ZeroDivisionError()
    return num1/num2
```

Try - Except

```
try:
    # The code that can possibly throw an exception
    add_something(1,3)
    add_something('3',4)
except:
    # Executed if an exception occurs
    print("An Exception was caught")
```



The **except** keyword is the Python equivalent of **catch**

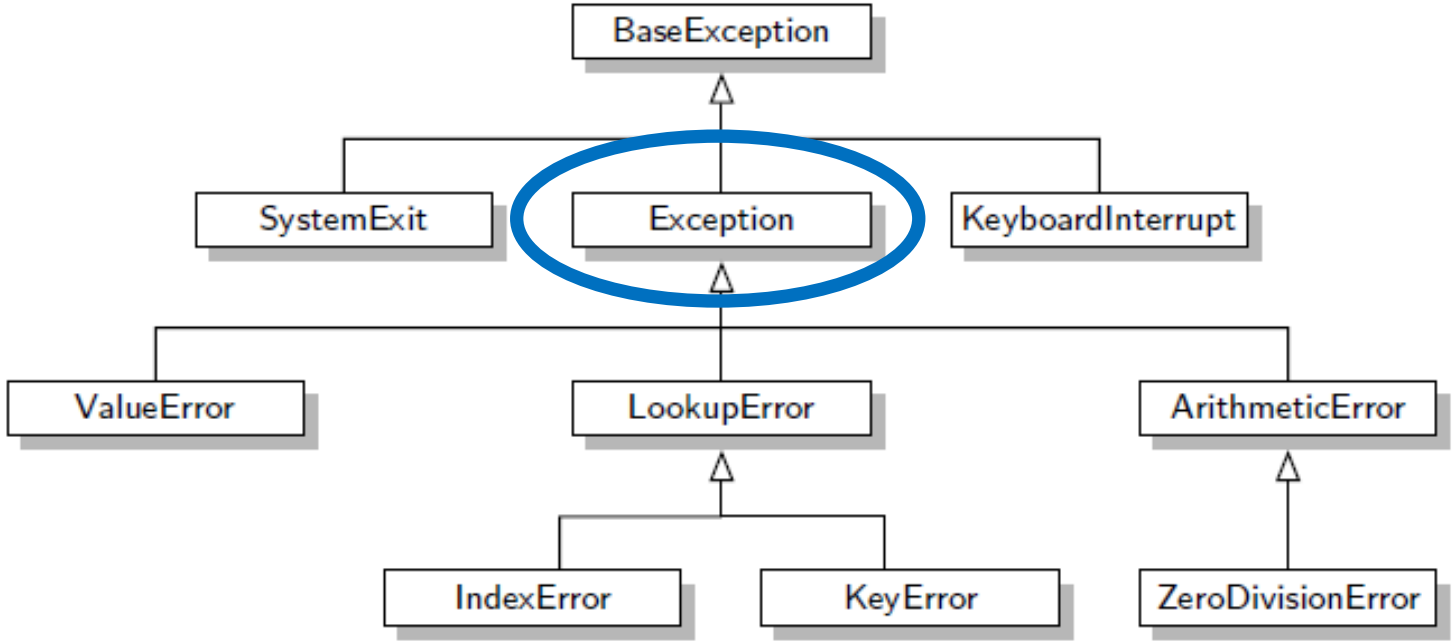
in Java/C++

This is how we deal with Dynamic Types

```
def add_something(val1, val2):

    if not isinstance(val1, int) or not isinstance(val2, int):
      raise TypeError("Invalid data type. Can only add integers")

    print(val1 + val2)

…

…

try:
    add_something(1,3)
    add_something('3',4)
except:
    print("An Exception was caught")
```

A Note on Exception Inheritance Hierarchy

- All Exceptions are objects that inherit from *BaseException*
- Most Exceptions we handle inherit from *Exception*, which inherits from *BaseException*.
- If we want an except clause that will handle any exception, we would try to handle the exception of type '*Exception*'

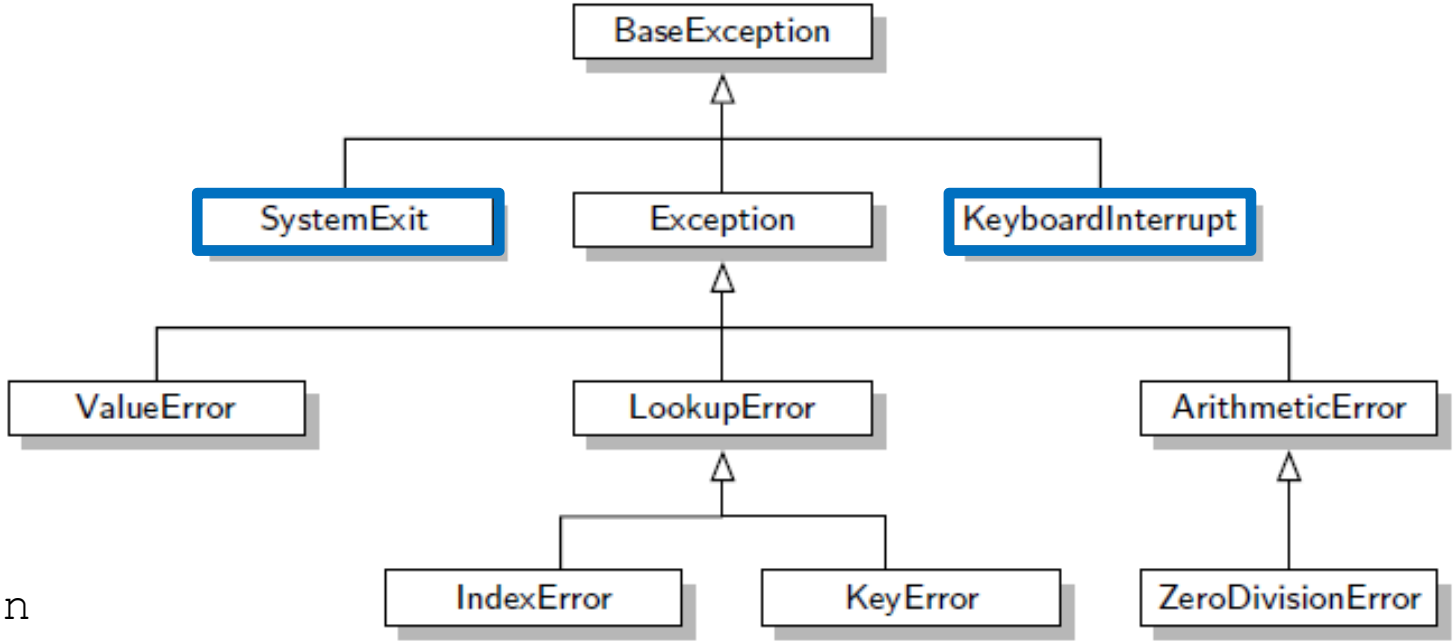


A Note on Exception Inheritance Hierarchy

- **SystemExit** and **KeyboardInterrupt** are special exception types that do not inherit from *Exception*. This is because we generally do not want to catch these exceptions. They are used to exit the program, or interrupt a program from the terminal.

- We should **NEVER** have an empty *except* clause since that would catch all the exceptions.

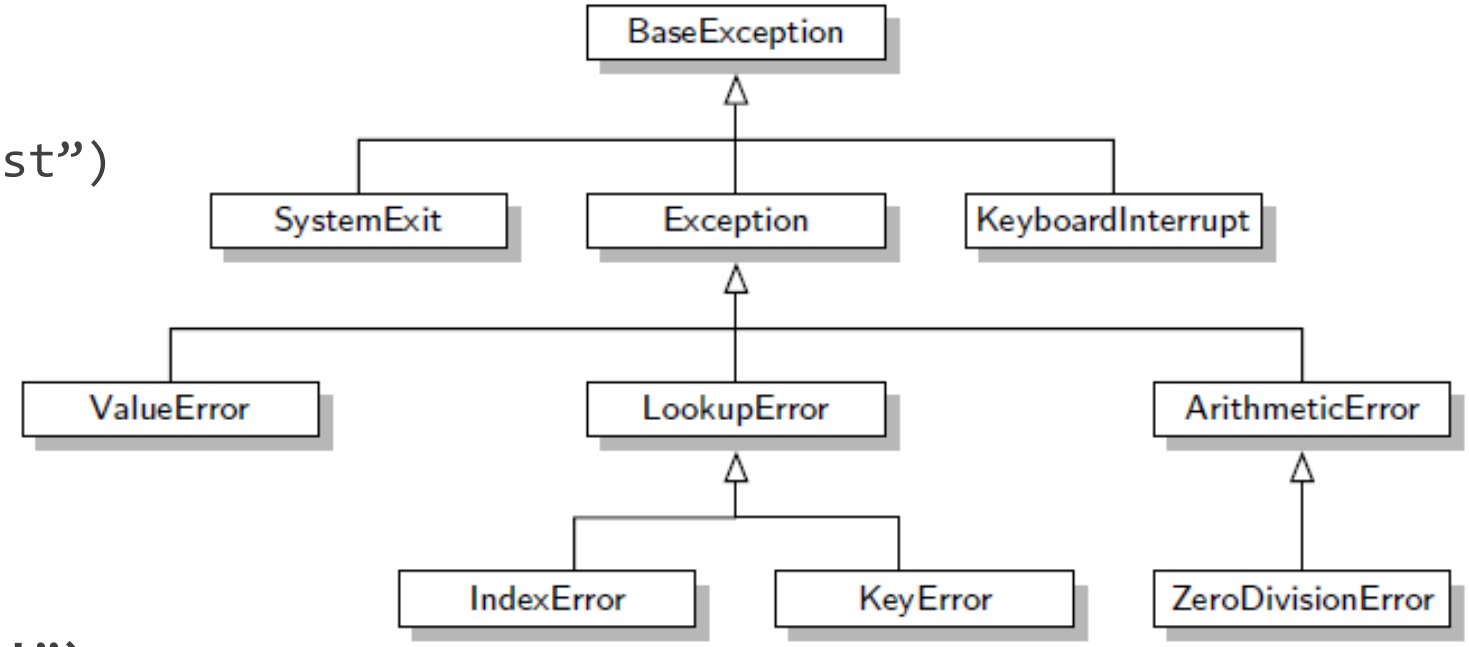
```
try:
    # code that throws exception
except:
    # catches all exceptions
```



A Note on Exception Inheritance Hierarchy

What would get printed?

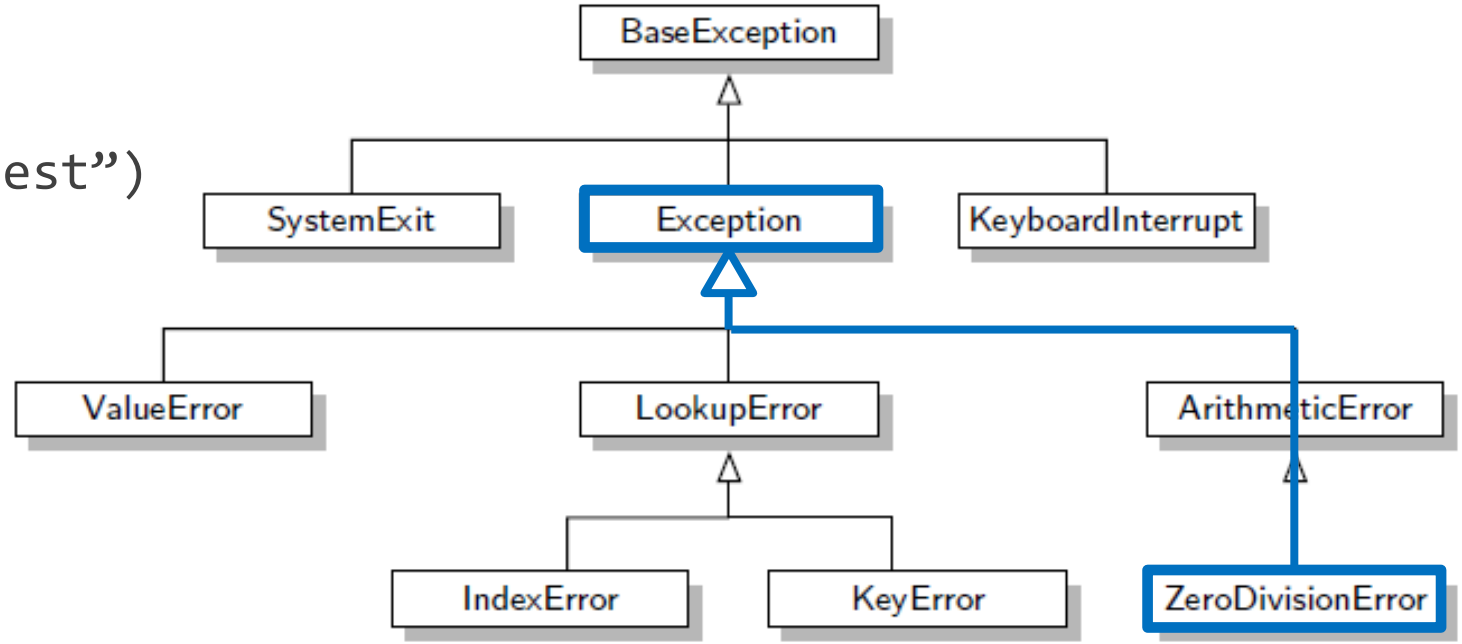
```
class Test():
    def some_method(self):
        raise ZeroDivisionError("Test")


test_obj = Test()

try:
    test_obj.some_method()
except Exception:
    print("Exception caught!")
except ZeroDivisionError:
    print("ZeroDivisionError caught!")
```



A Note on Exception Inheritance Hierarchy

What would get printed?

```
class Test():
    def some_method(self):
        raise ZeroDivisionError("Test")


test_obj = Test()
try:
    test_obj.some_method()
except Exception:
    print("Exception caught!")
except ZeroDivisionError:
    print("ZeroDivisionError caught!")
```



A Note on Exception Inheritance Hierarchy

What would get printed?

```
class Test():
    def some_method(self):
        raise ZeroDivisionError("Test")


test_obj = Test()
try:
    test_obj.some_method()
except Exception:
    print("Exception caught!")
except ZeroDivisionError:
    print("ZeroDivisionError caught!")
```

- **Order of catching exceptions matters**
- **ZeroDivisionError** inherits from Exception
- Therefore `except Exception` catches **ZeroDivisionError** since it is a subclass of Exception
- We should always catch the subclass versions of exceptions before wider parent exceptions
- In fact, PyCharm will warn you that except Exception is too broad

A Note on Exception Inheritance Hierarchy

What would get printed?

```
class Test():
    def some_method(self):
        raise ZeroDivisionError("Test")


test_obj = Test()
try:
    test_obj.some_method()
except ZeroDivisionError:
    print("ZeroDivisionError caught!")
except Exception:
    print("Exception caught!")
```

- The code on the left moves **ZeroDivisionError** higher up
- This allows us to properly get the more specific error message "ZeroDivisionError caught"

Try – Except – Else - Finally

```
try:
    statements
        ...
except:
    statements
        ...
else:
    statements
        ...
finally:
    statements
        ...
following_statement
```

Try

Run this as a normal part of the program

Except

Execute this when there is an exception

Else

Execute this only if no exceptions are raised

Finally

Always execute this

Easier To Ask For Forgiveness Than Permission

A Philosophy that Python has adopted.

Multiple If-Checks are bad (Look Before You Leap)

Assume code will work

Handle any exceptions/errors as they arise

Why are If-Checks bad?

- Exceptions are fast
- If-checks waste CPU cycles for situations that generally don’t occur often
- Keep our functions Atomic. They only do one thing. Let someone else deal with the exceptions.

Custom Exceptions

Since all exceptions are objects that inherit from Exception, we can define our own custom exceptions!

Make a **regular class** inherit from **Exception**

```
class MyException(Exception):
  ◦ #class code
```

Optional : Create an init with a custom error message

```
def __init__(self, my_msg):
    super().__init__(my_msg)
```

args and **as** in Exceptions

When catching an exception, we might want to get the **instance** of an exception to retrieve some data

This is done by adding the keyword **as**, and a **variable name** after the exception we just caught

```
except MyException as e:
    print(e.args[0])
```

This catches the exception into the instance variable **e**

After that we can access the exception like an instance of any other class

args and **as** in Exceptions

The BaseException's initializer contains an **args** to store a tuple of arguments

Typically there is only one argument, which is a string describing the error

However additional arguments can be accessed using [] on the args

```
try:
    raise MyException("Custom Error message", 999)
except MyException as e:
    print(e.args) #('Custom Error message', 999)
    print(e.args[0]) #Custom Error message
    print(e.args[1]) #999
```

lecture_exception_example.py, character_example.py

A Quick Recap: Unit Tests

What is Unit Testing?

We need to test our code often!

Part of **test driven development** involves writing our test cases first, and THEN writing the code.

Once code is written, we cross check it with our test cases to ensure it meets standards and works as intended.

Unit Tests

WHAT: A piece of code that exercises a very small, specific area of functionality of a given system.

HOW: A unit test invokes a particular function with a given input. It then asserts whether the output received matches a given expected output.

WHY?

It is an organized approach which helps break down our program into small bite sized pieces of functionality. Each chunk of functionality can be tested in isolation.

Unit Tests

Steps:

- Assemble – Create an environment in which the function can run
- Act – Run the code with a given input(s).
- Assert – Compare the result

Rules:

- A collection of unit tests
- Each test is a function that contains an **assertion**
- Assertions must test individual functions of your code
- The unit test functions go inside the test class
- **The functions must all begin with <u>test</u>**

Assertions

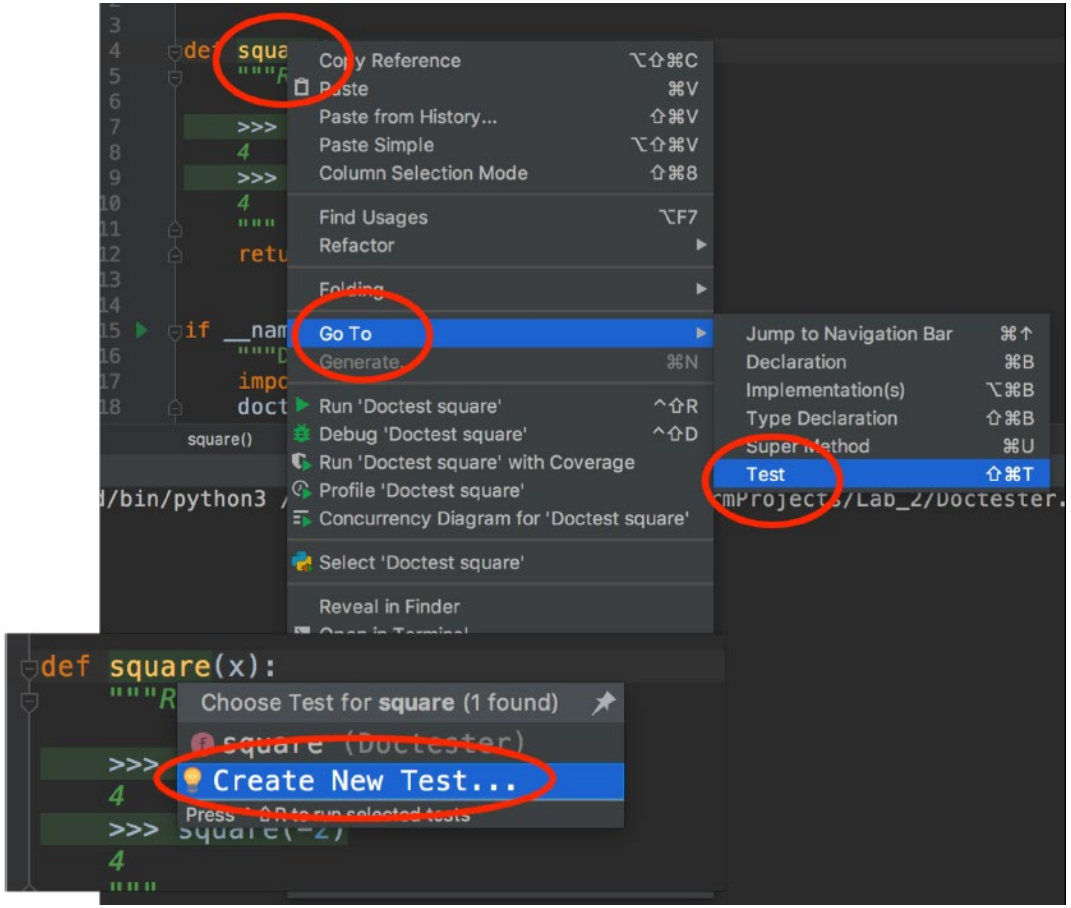
Method	Checks that	New in
`assertEqual(a, b)`	`a == b`	
`assertNotEqual(a, b)`	`a != b`	
`assertTrue(x)`	`bool(x) is True`	
`assertFalse(x)`	`bool(x) is False`	
`assertIs(a, b)`	`a is b`	3.1
`assertIsNot(a, b)`	`a is not b`	3.1
`assertIsNone(x)`	`x is None`	3.1
`assertIsNotNone(x)`	`x is not None`	3.1
`assertIn(a, b)`	`a in b`	3.1
`assertNotIn(a, b)`	`a not in b`	3.1
`assertIsInstance(a, b)`	`isinstance(a, b)`	3.2
`assertNotIsInstance(a, b)`	`not isinstance(a, b)`	3.2

Unit testing in PyCharm

Right click on the function we want to test.

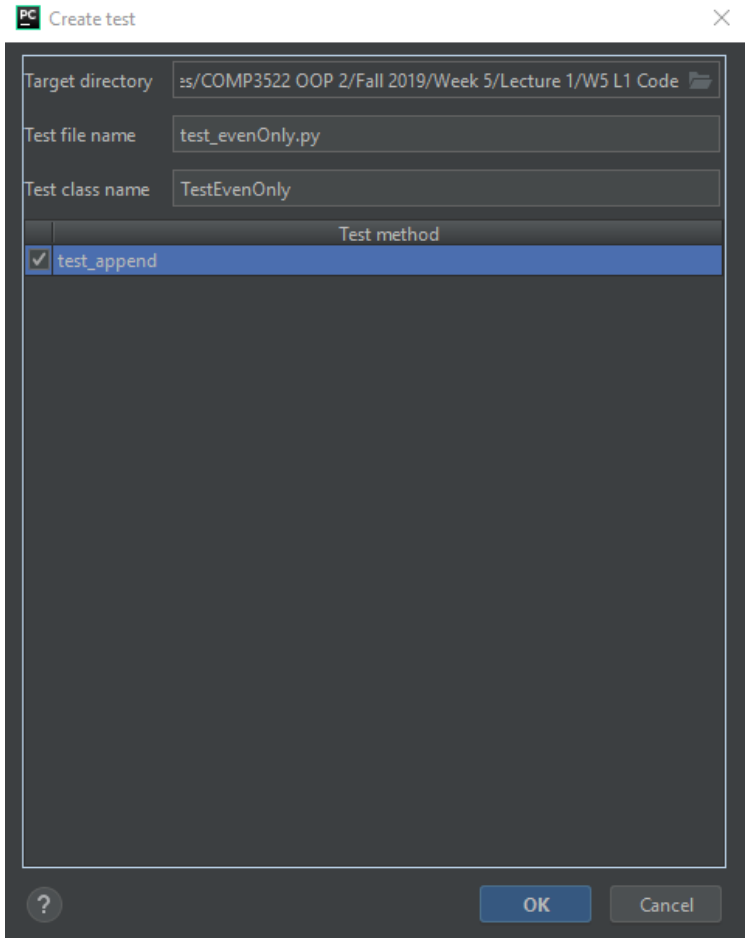
Select Go To -> Test

In the dialog select Create New Test if this is the first unit test for that function



Unit testing in PyCharm

Make sure your function is checked



Unit testing in PyCharm

```
#created by default
class Test(TestCase):
    def test_sum(self):
        self.fail()
```

```
#after edit
class Test(TestCase):
    def test_sum(self):
        self.assertTrue(MyClass.sum(1, 2)==3)
```

A new Test class will be generated for you, along with a new test function

The generated test function will be **set to fail by default**. You will need to write code to call your code and create assertions that evaluate to true

To create additional test cases, just add additional “def test...” functions to the Test class

Note you MUST begin functions with the word **test** or they will not appear as runnable tests

calculator.py, test_calculator.py

File Handling

Files

To work with a file in Python, the following sequence needs to be followed

1. Open a file – Specify the **encoding**, file **mode** and file **name**
2. Conduct File Operations – Read, Write, Append
3. Close the file – Release memory and tie up any loose ends.

Python Docs:

https://docs.python.org/3/tutorial/inputoutput.html

File Mode	Description
r	Opens a file for **reading** only. The file pointer is placed at the beginning of the file. This is the default mode.
rb	Opens a file for **reading** only in binary format. The file pointer is placed at the beginning of the file. This is the default mode.
r+	Opens a file for both **reading and writing**. The file pointer placed at the beginning of the file.
rb+	Opens a file for both **reading and writing** in binary format. The file pointer placed at the beginning of the file.
w	Opens a file for **writing** only. **Overwrites the file if the file exists**. **If the file does not exist, creates a new file for writing**.
wb	Opens a file for **writing** only in binary format. **Overwrites the file if the file exists**. **If the file does not exist, creates a new file for writing**.
w+	Opens a file for both **writing and reading**. **Overwrites the existing file if the file exists**. **If the file does not exist, creates a new file for reading and writing.**
wb+	Opens a file for both **writing and reading** in binary format. **Overwrites the existing file if the file exists**. **If the file does not exist, creates a new file for reading and writing.**
a	Opens a file for **appending**. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. **If the file does not exist, it creates a new file for writing.**
ab	Opens a file for **appending** in binary format. The file pointer is at the end of the file if the file exists. That is, the file is in the append mode. **If the file does not exist, it creates a new file for writing.**
a+	Opens a file for both **appending and reading**. The file pointer is at the end of the file if the file exists. The file opens in the append mode. **If the file does not exist, it creates a new file for reading and writing.**
ab+	Opens a file for both **appending and reading** in binary format. The file pointer is at the end of the file if the file exists. The file opens in the append mode. **If the file does not exist, it creates a new file for reading and writing.**

File – Example

```
#Step1: Open a file
my_text_file = open("sample.txt", mode='r', encoding='utf-8')

#Step2: File operations
data = my_text_file.read()
print(f"File Data:\n{data}")

#Step3: Close the file
my_text_file.close()
```

With keyword

Manually closing files is not good practice

In the event of an **exception**, the program will halt execution and the **close() function will not be executed**.

To circumvent this, Python implements a **`with`** keyword.

The **with** keyword defines an **identifier** which **represents an object** like a file, an active connection to a server, anything that needs to be initialized and de-initialized.

To adhere to this protocol for user defined types, the `__enter__(self)` and `__exit__(self)` methods need to be implemented

Refer to PEP 345 for more information:

https://www.python.org/dev/peps/pep-0343/

With keyword - Example

Syntax:

```
with EXPR as VAR:
    BLOCK1
```

Example:

```
with open("test.txt", mode = 'w',encoding = 'utf-8') as f:
    f.write("my first file\n")
    f.write("This file\n\n")
    f.read() #This will cause an exception. The file will close safely
    f.write("contains three lines\n")
```

Using with vs not using with

```
# Not using with
my_text_file = open("sample.txt", mode='r', encoding='utf-8')

data = my_text_file.read()
print(f"File Data:\n{data}")

my_text_file.close()
```

```
# Using with

with open("sample.txt", mode='r', encoding='utf-8') as my_text_file:

      data = my_text_file.read()
      print(f"File Data:\n{data}")
```

Seek

```
with open("sample.txt", mode='r') as my_text_file:

    data = my_text_file.read()
    print(f"File Data:\n{data}")

    # This wont print anything. File pointer is at the end of the file
    data = my_text_file.read()
    print(f"Printing Data again: {data}")

    # Need to seek back to the beginning for multiple reads
    my_text_file.seek(0)
    data = my_text_file.read()
    print(f"Printing data after seeking to the beginning:\n
          {data}")
```

file_handling_example.py

Load json from file

Import json module

Use with open just like a text file

Use json.load(identifier) to convert data into a dictionary

```
import json

with open("data.json", mode='r', encoding='utf-8') as data_file:
    data = json.load(data_file) #converts json format to python dictionary
    print(data)
```

Write to json file

Import json module

Open a new person.json file, set to **'w'**rite mode

- Open file as json_file

Convert dictionary into json, then write json file out

```
import json
person_dict = {"name": "Homer", "children": ["Bart", "Lisa","Maggie"],
"married": True, "fav_food": 'Donut' }

with open('person.json', 'w') as json_file:
  json_data = json.dumps(person_dict) # convert dictionary to json format
  json_file.write(json_data) # write out json_data to file
  # json.dump(person_dict, json_file) #same as above 2 lines
```

Function Overloading and Parameters

WEEK 4 PART 2

Overloading

In Python, function/method overloading doesn't quite work like Java.

Since Python is dynamically typed, we can't have a scenario like this

```
def add(int_a, int_b):
    return int_a + int_b

def add(string_a, string_b):
    return string_a + string_b
```

Same function!

```
def add(a, b):
    return a + b
```

These are exactly the same function! We can't differentiate based on parameter data types.

Overloading

Python does NOT support traditional “Function overloading“

“Function overloading“ in Python refers more to the flexibility of function signatures and dynamic typing to achieve similar effects.

We can implement Function/Method Overloading in the following ways:

- **Using Default Arguments**
- **Using Named Arguments**
- **Using Variable List Arguments**
- **Using Keyword Arguments**

But first! Positional Arguments

These are the standard functions we are used to.

- The parameters have fixed positions
- Parameters must be given in the order expected.

Eg:

```
def add_vectors(vector1, vector2):
    x = vector1.x + vector2.x
    y = vector1.y + vector2.y
    z = vector1.z + vector2.z
    return Vector(x, y, z)

position = …
velocity = …
add_vectors(position, velocity)
```

Default Arguments

Just like Java and C++ we can allow our function parameters to have default arguments.

Default arguments need to be at the **end of the argument list**.

```
def __init__(team_number, team_name = “”):
```

You cannot have a positional / non-default argument after a default argument.

```
def __init__(team_name = “”, team_number): #wrong order!
```

The least likely default arguments should be to the left of the most likely default arguments.

Default arguments

```
class Team:

def __init__(self, team_number, team_name = “”, member_list = None):

    self.team_number = team_number

    self.name = team_name

    self.members = member_list


assignment1_team1 = Team(0) #Team(0,””,None)

assignment1_team2 = Team(1, “Pugs 4 Life”) #Team(1,”Pugs 4 Life”,None)

assignment2_team3 = Team(2, “Tamagotchi App Developers”, [“Jenkins”,
“Natalie”, “Susan”])

#Team(2,”Tamagotchi App Developers”, [“Jenkins”, “Natalie”, “Susan”])
```

Remember! Python is interpreted!

What happens if we write a program like this?

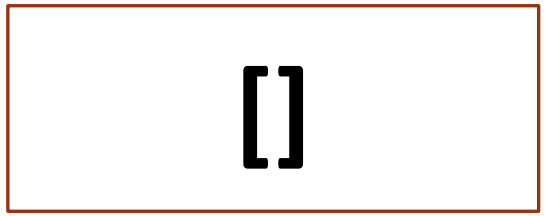
Anything we provide to a default argument **is evaluated when the function is first interpreted, not when it is called**. That is, we **can't have dynamically generated default values**.

The **empty list parameter** is created once.

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```



Default - message

A single message variable is created when code is interpreted

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

[‘Hi’]

Default - message

The first function call appends ‘Hi’ to that single default instance of message

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

['Hi', **'Hi'**]

Default - message

The second function call appends another 'Hi' to that single default instance of message

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

['Hi', 'Hi']

Default - message

['new-1']

new_1

new_1 is a new list that is NOT using the default instance of message. New memory is allocated

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

['Hi', 'Hi']

Default - message

['new-1', **'Hi'**]

new_1 message

Message parameter is pointing at new_1. Appends 'Hi' to new_1 list

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

['Hi', 'Hi']

Default - message

['new-1', 'Hi']

new_1

['new-2']

new_2

new_2 is a new list that is NOT using the default instance of message. New memory is allocated

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

[‘Hi’, ‘Hi’]

Default - message

[‘new-1’, ‘Hi’]

new_1

[‘new-2’, **‘Hi’**]

new_2 **message**

Message parameter is pointing at new_2. Appends ‘Hi’ to new_2 list

Remember! Python is interpreted!

```
def print_message(message=[]):
    message.append('Hi')
    print(message)

print_message()
print_message()
new_1 = ['new-1']
print_message(new_1)
new_2 = ['new-2']
print_message(new_2)
print_message()
```

['Hi', 'Hi', **'Hi'**]

Default - message

['new-1', 'Hi']

new_1

['new-2', 'Hi']

new_2

Again using default parameter message, so 'Hi' appending to default message list

Default arguments reminders

Be careful when using default arguments!

A new dynamic parameter is not generated every time the default parameter is used. The same default parameter is used every time

The behavior is different when operating on mutable vs immutable parameters in a function

team_default_arguments.py

Named Arguments: Changing the Order

If we have named arguments, we can change the order of parameters in the function call by referring to its name.

Positional arguments

```
def my_func(a_list, a_num, a_string):
    print(a_list, a_num, a_string)
```

```
my_func(['cat','dog'], 5, 'hello')
```

The order that parameters passed in matters with **positional arguments** (this is the one we're used to). Errors will occur otherwise

Named Arguments: Changing the Order

```
def my_func(a_list, a_num, a_string):
    print(a_list, a_num, a_string)
```

```
my_func(a_num=5, a_string='hello', a_list=['cat','dog'])
```

The **position of parameters that are passed in doesn't matter with named arguments**. All that matters is explicitly assigning a value to a matching parameter name

Named Arguments: Changing the Order

```
def my_func(a_list, a_num=0, a_string=''):
    print(a_list, a_num, a_string)


my_func(a_num=5, a_list=['cat','dog'])
```

This is especially useful when functions have **default parameters**. Don't need to pass in all parameters. Only need to pass in parameters we care about

Variable List Arguments

What if we wanted to provide a variable number of parameters?

E.g:

```
my_print("COMP3522", "is great.", "Tuesdays are long")
my_print("I love", "weekends!")
```

We could have the function take a **list** parameter and iterate through it.

Or maybe there is a better way to do this since Python is cleaner and nicer?

Pass unknown number of arguments

Can pass variable list of arguments into function using **list**

Wrap all arguments in list **[]**

```
def my_print(list_of_items):
    for item in list_of_items:
        print(item)

my_print(['dog', 5, {1:'hi', 2:'hello'}])
```

Variable Argument List - “Packing Arguments”

We specify an argument with an * preceding its name. The standard/convention is *args

This is known as **packing** arguments. All arguments are packed into a **tuple** named args

```
def my_print(*args):
    for item in args:
        print(item)

my_print('dog', 5, {1:'hi', 2:'hello'})
```

Variable Argument List - “Unpacking”

The * (asterisk) can be used with **containers** as well. This is known as **unpacking**

In the example below, the **list** is “unpacked” into the individual string, integer, and list objects

```
def my_print(in_str, in_int, in_dict):
    print(in_str, in_int, in_dict)

my_list = ['dog', 5, {1: 'hi', 2: 'hello'}]
my_print(*my_list) #unpack list
```

team_variable_arguments.py

Variable Argument List - “Unpacking”

Can combine positional arguments, default arguments, and variable argument list.

Good practice to have positional arguments first

```
def my_function(a, b=10, *args, c=20):
    print(f"a: {a}")
    print(f"b: {b}")
    print(f"args: {args}")
    print(f"c: {c}")

my_function("Hello", 2, 3, 4, c = 99)
```

```
Output:

a: Hello
b: 2
args: (3, 4)
c: 99
```

Recap

- Python Functions
  - Positional Arguments
  - Default Arguments
  - Named Arguments
  - Variable List Arguments
  - **Implicit Unpacking**
  - **Variable Keyword Aguments**

Implicit Unpacking

Unpacking

We’ve learned about unpacking using the * asterisk

There’s another form of unpacking that occurs implicitly without using the operator

Unpacking

Try this code:

```
item1,item2 = (1, 2)
print(item1)
```

1

```
print(item2)
```

2

Dynamically assigning each element of a sequence to a corresponding variable is known as **unpacking**.

Unpacking

Example of unpacking dictionary

```
default_character_data = {
        'name' : "Untitled",
        'level' : 4,
        'max_level' : 10,
        'weapon' : "Sword",
        'Gender' : "Male"
    }
for a_key, an_item in default_character_data.items():
    print(f"{a_key} - {an_item}")
```

Unpacking

Example of unpacking dictionary

```
{'name' : "Untitled",
'level' : 4,
'max_level' : 10,
'weapon' : "Sword",
'Gender' : "Male“}
```

```
for a_key, an_item in default_character_data.items():
    print(f"{a_key} - {an_item}")
```

Variable Keyword Arguments

Variable Keyword Arguments

Named Arguments are also known as **Keyword Arguments**.

Just like we can accept an arbitrary variable number of arguments using **`*args`**

`*args` is treated like a sequence type

We can accept an arbitrary variable number of keyword arguments using **`**kwargs`**

`**kwargs` is treated like a dictionary

Recall: Named Arguments

```
def my_func(a_list, a_num, a_string):
    print(a_list, a_num, a_string)
```

```
my_func(a_num=5, a_string='hello', a_list=['cat','dog'])
```

The position of parameters that are passed in doesn’t matter with named arguments. All that matters is explicitly assigning a value to a matching parameter name

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

Looks like we're passing in **named arguments**

Also notice how this looks like a dictionary without the **{}**

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

There are keys

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

There are keys, and **values**

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

****kwargs** interprets the key/value arguments passed in as a dictionary

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

kwargs is a dictionary, so in order to get the key/value pairs, we need to explicitly call the **items method**

Notice how we're using *unpacking* to get the key/value pairs from the dictionary

Variable Keyword Arguments

```
def variable_keyword_args(**kwargs):
    for key, value in kwargs.items():
        print(key, value)

variable_keyword_args(port=21, host='localhost', debug=True)
```

- Each key/value pair can be extracted using **key**, **value** variables
- Note that key and value are NOT special names, they could be replaced by x, y and the code behaves the same

variable_keyword_arguments.py